Question 3

Q: An image shows aliasing artifacts. Identify the cause using sampling and quantization concepts and propose a corrective approach.

Theoretical Concept: Aliasing Artifacts

Aliasing occurs when the sampling rate is less than twice the highest frequency present in the image, violating the **Nyquist-Shannon Theorem**. This causes high-frequency details (like fine patterns or edges) to appear as jagged lines or Moire patterns.

Corrective Approach: Apply an anti-aliasing (low-pass) filter, such as a Gaussian Blur, *before* sampling or downsizing the image to remove high frequencies that cannot be properly captured.

💻 OpenCV Python Solution

Copy

```
import cv2  # Import OpenCV for image processing algorithms
import numpy as np                                          # Import Numpy for mathem
import matplotlib.pyplot as plt                             # Import Matplotlib to vi

# Step 1: Generate a high-frequency grid pattern
img = np.zeros((200, 200), dtype=np.uint8)  # Create a blank black image array filled
img[::5, :] = 255
img[:, ::5] = 255

# Step 2: Apply a Gaussian Blur to act as an anti-aliasing (Low-Pass) filter
# The (5,5) kernel removes high frequencies before downsampling
blurred = cv2.GaussianBlur(img, (5, 5), 0)  # Apply Gaussian smoothing to reduce high-

# Step 3: Now we can safely downsample the image
# cv2.INTER_AREA is mathematically designed to prevent aliasing
downsampled = cv2.resize(blurred, (40, 40), interpolation=cv2.INTER_AREA)  # Resize im
bad_downsample = cv2.resize(img, (40, 40), interpolation=cv2.INTER_NEAREST)  # Resize
```

```
# Step 4: Show the difference
plt.figure(figsize=(8,4))  # Create a new figure window for display
plt.subplot(121), plt.imshow(bad_downsample, cmap='gray'), plt.title('Aliased (Jagged)
plt.subplot(122), plt.imshow(downsampled, cmap='gray'), plt.title('Anti-Aliased (Smoot
plt.show()  # Show the final plot window to the user
```

🎯 Expected Output

You will see two small grids. The left one (Aliased) will look distorted, broken, and jagged. The right one (Anti-Aliased) will be slightly soft but structurally accurate.

🚀 How to Execute & Submit

1. Google Colab Execution:

- Open Google Colab and create a New Notebook.
- Click the 📋 **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File** > **Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.